

Small variations make a big difference

Oct 12, 2025

This article contains spoilers for [Codeforces Round 1053 \(Div. 1\) Problem C](#) and [Codeforces Round 1058 \(Div. 1\) Problem D1](#).

Have you ever been stuck on a problem, only to look up the solution and find out that their approach is basically the same as yours? Somehow their setup led to a solution, and yours didn't.

This has happened to me a lot. In the past, I've written these situations off (e.g., "so close, I had all the right ideas!" or "I was just unlucky!"). But that mindset didn't help me improve. In this blog, I'll explain what did.

Example 1: [Codeforces Round 1058 \(Div 1.\) Problem D1](#)

A real application of the technique! I solved this in today's contest by thinking in the "right" way.

The gist is that there's a $\Theta(n^2)$ DP that you have to optimize:

Let $v_1, \dots, v_n$ be the array, with $v_1 < \dots < v_n$ (see the editorial for why we can make this assumption without loss of generality).

Then, define $dp(i)$ to be the min cost to partition the prefix $v_1, \dots, v_i$. We have $dp(0) = 0$ and

$$dp(i) = \min_{0 \leq j < i} \left(dp(j) + \left\lceil \frac{v_i}{v_{j+1}} \right\rceil \right).$$

for every $1 \leq i \leq n$. The answer to the problem is $dp(n)$.

Part 2.3 of the [editorial](#) describes a CHT optimization with some extra details (monotonic stack and rollbacks on a Li-Chao Tree - I have no idea what this means).

But with a very small change, the implementation becomes extremely easy. Instead let $dp(i)$ be the cost of the **suffix** $v_i, \dots, v_n$, so now

$$dp(i) = \min_{i < j \leq n+1} \left(dp(j) + \left\lceil \frac{v_{j-1}}{v_i} \right\rceil \right).$$

Now the CHT is extremely simple - see [my comment](#).

Why did this happen? Well, in the first version, having v_{j+1} in the denominator of a ceiling division is really inconvenient, since the denominator changes as j changes. In the second version, v_i is in the denominator, which makes the ceiling division really easy to handle (for each value, we divide by the same thing).

Key point: In general, if you have some symmetry in the setup (here, DP on prefixes vs suffixes), you should be open to trying both choices, even if one appears more natural. Some later step in the solution (here, the roles of i and j in CHT) may not share that same symmetry.

In contest, I considered the "less natural" DP on suffixes, and it paid off.

Example 2: [Codeforces Round 1053 \(Div. 1\) Problem C](#)

This is an earlier contest, where I locked in a choice based on what seemed most natural to me, and it backfired.

Once again, there's a $\Theta(n^2)$ DP to optimize:

Reinterpret the scenario as follows: Alice and Bob each go to the shop n times, in some order. On Alice's i th visit, she buys item a_i if it exists, or nothing if it was already taken. Similarly, on Bob's j th visit, he buys item b_j if it exists, and nothing otherwise.

Let $dp(i, j)$ be the maximum value for Alice after she makes i visits and Bob makes j visits. Then, the answer to the problem is $dp(n, n)$, and we have the recurrence

$$dp(i, j) = \min \begin{cases} dp(i-1, j) + [a_i \notin \{b_1, \dots, b_j\}] \cdot v_{a_i}, \\ dp(i, j-1). \end{cases}$$

for $1 \leq i, j \leq n$ (there are also some base cases that I'll ignore). Here, $[cond]$ is 1 if $cond$ is true, and 0 otherwise.

From here, you'll want to permute the arrays so that $b_1 = 1, b_2 = 2, \dots, b_n = n$, because that converts the complicated condition $a_i \notin \{b_1, \dots, b_j\}$ to the simple $a_i > j$. It's so simple that you can compute an entire row of dp values with a few segment tree operations (see [editorial](#)).

But I never got there, because I made the wrong choice of letting $dp(i, j)$ be the "penalty" of the items taken so far: the penalty is the sum of the negative values taken by Alice and the positive values taken by Bob. Somehow I thought this "balanced" approach would be better. I got another $\Theta(n^2)$ DP:

$$dp(i, j) = \min \begin{cases} dp(i-1, j) + [a_i \notin \{b_1, \dots, b_j\}] \cdot \max(0, -v_{a_i}), \\ dp(i, j-1) + [b_j \notin \{a_1, \dots, a_i\}] \cdot \max(0, v_{b_j}). \end{cases}$$

but this is much uglier and not possible to optimize using the $b_i = i$ trick.

There are also other variations on the setup: what if you compute the minimum value for Bob? What if you make $a_i = i$ instead? These all affect the eventual optimization you have to perform.

If you found the "correct" setup first, you might think this is all silly - of course keeping track of Alice's value is easier.

But noticing these differences is the point! Sometimes it won't be obvious which choice to make, so being aware of your options and their tradeoffs is the difference between guaranteeing a solve and blindly going down one path.

Example 3: an ODE from MATB44

This example is quite different. Credits to Stefanos Aretakis for explaining integrating factors in this way during lecture.

Consider the ODE

$$(e^x - \sin y)dx + \cos y dy = 0.$$

If you've heard about exact ODEs, then you'll know that if the ODE meets some special condition ($\frac{\partial P}{\partial y} = \frac{\partial Q}{\partial x}$, where $P = e^x - \sin y$ and $Q = \cos y$), then there's a simple way to find a solution (not worth describing here).

The equation doesn't satisfy the special condition. But, if you multiply everything by e^{-x} , it does! And the solutions are exactly the same.

Why did this variation make a difference? Well, even though multiplying didn't affect the solutions, it did change the derivatives (in a nontrivial way, from the product rule). So we should expect that multiplying makes *some* kind of difference, even if we aren't lucky enough to get the condition we want.

(It might seem like a one-off, but this trick is always "doable", if you can find the right thing to multiply by. See <https://math.stackexchange.com/a/101694> for more details.)